



OKU ROUTER

Security Review



JANUARY, 2026

www.chaindefenders.xyz
<https://x.com/DefendersAudits>

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details
 - Commit Hash
 - Scope
 - Roles
5. Executive Summary
 - Issues Found
 - Issues Statuses
 - Security Grade
6. Findings
 - Medium
 - Low
 - Informational
 - Gas

Protocol Summary

Oku Router is an EVM-compatible smart routing contract that facilitates secure token swaps via external DEX aggregators. It validates backend-generated trade warrants, handles user approvals via Permit2 or legacy permits, applies protocol fees, and executes optimal swap routes by forwarding calls to supported aggregators (e.g. 1inch, Odos, Paraswap), which in turn interact with underlying DEXs. The router is designed to be chain-agnostic and serves as the on-chain execution and security layer for Oku's swap infrastructure.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

7e1747c2b1a91fd39274d569a9497a815c2e68ed

Scope

Id	Files in scope
1	./contracts/OkuRouter.sol
2	./contracts/routers/BaseAggregator.sol
3	./contracts/libraries/CanoeHelper.sol
4	./contracts/libraries/PermitHelper.sol
5	./contracts/libraries/SafeTransferLib.sol

Roles

Id	Roles
1	Owner
2	User

Executive Summary

Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	1	Significant risks
Low	2	Minor issues with low impact
Informational	2	Best practices or suggestions
Gas	3	Optimization opportunities

Issues Statuses

Id	Title	Status
M-01	Signature Replay Attack	Fixed
L-01	Zero Address Being A Valid Signer Can Lead To A Full Warrant Verification Bypass	Acknowledged
L-02	Two-Step Ownable Pattern	Fixed
I-01	'nonReentrant' Modifier Not Properly Ordered	Fixed
I-02	Allowance Assumption May Break With Partial-Spend DEXs	Acknowledged
G-01	Insufficient 'msg.value' Will Lead To Wasted Gas Due To Missing Early Check	Fixed
G-02	Redundant Recovered Signer Address Zero Check	Fixed
G-03	Missing Permit Deadline Pre-Check	Acknowledged

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is **97.00 / 100.00**. This reflects an excellent security posture with almost no room for improvement.

Findings

Medium

Mid 01 Signature Replay Attack

Location

CanoeHelper.sol

Description

The `CanoeHelper::verifyWarrant` function verifies an EIP-712 signed warrant but does not enforce nonce uniqueness or invalidate used nonces on-chain.

Although a `nonce` field (uint160) is included in the signed `packedValidationData` and thus cryptographically bound to the signature, there is no on-chain check or storage update to ensure that a given nonce (or warrant) has not already been used.

As a result:

- Any successfully verified warrant can be replayed multiple times by the same user or a malicious third party (as long as `block.timestamp` is still within `[validAfter, validBefore]`)
- Each replay executes the same swap parameters (`dataHash`), forwards the same `msg.value`, takes fees again, and interacts with the external aggregator target again
- There is no mechanism to mark a nonce as consumed after successful verification

Proof of Concept

1. Off-chain quote engine generates a warrant for user Alice:
 - `nonce = 12345`
 - `validAfter = block.timestamp`
 - `validBefore = block.timestamp + 300` (5 minutes)
 - `dataHash = keccak256` of quote params (`sellToken`, `buyToken`, `target`, `feeAmount`, etc.)
 - signed by `verifyingSigner`
2. Alice submits the transaction -> `fillQuoteTokenToToken( ... )` (or similar) -> warrant verifies -> swap executes successfully -> fees taken, tokens sent, event emitted.
3. Attacker (or even Alice herself, maliciously) re-submits the exact same transaction call-data (same Warrant struct, same signature, same everything):
 - `verifyWarrant` passes again because:
 - signature still recovers to `verifyingSigner`
 - timestamps still valid
 - nonce is not checked against any used storage
 - Swap executes again -> aggregator called again -> fees taken again -> tokens/ETH transferred again
4. This can be repeated many times within the time window.

Recommendation

Implement on-chain nonce validation and invalidation to ensure each warrant can only be used once.

1. Add storage in `BaseAggregator` (or a separate contract if preferred):

```
1 mapping(address => uint256) public userNonces;
```

2. Modify each `fillQuote ...` function (after successful `verifyWarrant`):

```
1 // After CanoeHelper.verifyWarrant( ... ) succeeds
2 require(warrant.nonce == userNonces[msg.sender], "Invalid nonce");
3
4 userNonces[msg.sender]++; // consume nonce
```

Status

Fixed

Low

Low 01 Zero Address Being A Valid Signer Can Lead To A Full Warrant Verification Bypass

Location

OkuRouter.sol:72

Description

The `CanoeHelper::verifyWarrant` function uses `address(0)` as a special sentinel value to disable signature verification. When `warrant.verifyingSigner == address(0)`, the function immediately returns without performing any cryptographic signature validation. This design allows for disabling warrant checks under specific conditions.

However, the `OkuRouter::updateValidSigner` function lacks validation to prevent the owner from accidentally adding `address(0)` as a valid signer. If the owner mistakenly calls `updateValidSigner(address(0), true)`, the zero address would be marked as a valid signer. Subsequently, any user could craft a transaction with a warrant struct containing `verifyingSigner = address(0)`, which would bypass all signature verification checks in the contract. This could allow unauthorized

parties to execute swaps without proper authorization, completely circumventing the cryptographic security mechanism intended to protect the protocol.

Recommendation

Implement a validation check in the `updateValidSigner` function to prevent `address(0)` from being added as a valid signer:

```
1 function updateValidSigner(address target, bool add) external onlyOwner {
2     require(target != address(0), "CANNOT_ADD_ZERO_ADDRESS_AS_SIGNER");
3
4     validSigners[target] = add;
5     if (add) {
6         emit ValidSignerAdded(target);
7     } else {
8         emit ValidSignerRemoved(target);
9     }
10 }
```

Alternatively, if there's a legitimate use case for disabling signature checks via `address(0)`, emit a specific warning event when this occurs to provide explicit tracking and intent:

```
1 event ValidSignerZeroAddressOperation(bool add);
2
3 function updateValidSigner(address target, bool add) external onlyOwner {
4     if (target == address(0)) {
5         emit ValidSignerZeroAddressOperation(add);
6     }
7
8     validSigners[target] = add;
9     if (add) {
10         emit ValidSignerAdded(target);
11     } else {
12         emit ValidSignerRemoved(target);
13     }
14 }
```

The first approach (preventing `address(0)`) is recommended as it eliminates the risk entirely.

Status

Acknowledged

Low 02 Two-Step Ownable Pattern

Location

OkuRouter.sol

Description

The contract uses a single-step ownership transfer pattern. With this approach, ownership is immediately transferred to a new address once `transferOwnership` is called. If an incorrect or unintended address is provided, ownership can be permanently lost.

A two-step ownership transfer pattern is generally considered best practice, as it requires the new owner to explicitly accept ownership before the transfer is finalized.

Recommendation

Adopt a two-step ownership transfer mechanism (e.g., `Ownable2Step`) to reduce the risk of accidental or irreversible ownership transfers and to align with established best practices.

Status

Fixed

Informational

Info 01 nonReentrant Modifier Not Properly Ordered

Location

BaseAggregator.sol

Description

The `nonReentrant` modifier is currently positioned after the `whenNotPaused` modifier in external functions. However, best practice dictates that the `nonReentrant` modifier should be placed first among all modifiers. This is because the `nonReentrant` modifier sets a reentrancy guard flag (`status = 2`) at the beginning of execution. If other modifiers are evaluated before `nonReentrant`, those modifiers

could potentially execute code (including external calls) before the reentrancy guard is in place. This leaves a window of vulnerability where a reentrancy attack could theoretically be launched during the execution of the preceding modifiers.

Recommendation

Reorder all function modifiers to place `nonReentrant` as the first modifier immediately after the function visibility and state mutability specifiers (e.g., `external`, `payable`). The correct order should be:

```
1 function fillQuoteEthToToken( ... )
2     external
3     payable
4     nonReentrant
5     whenNotPaused
6     onlyApprovedTarget(target)
7     onlyApprovedSigner(warrant.verifyingSigner)
8 { ... }
```

This ensures the reentrancy guard is established before any other modifier code executes, providing comprehensive protection against reentrancy attacks across all functions that use this modifier.

Status

Fixed

Info 02 Allowance Assumption May Break With Partial-Spend DEXs

Location

BaseAggregator.sol

Description

In `BaseAggregator`, after each swap the contract checks whether the allowance of the swapped token has been fully consumed (i.e., allowance equals zero). This assumption can break if an integrated DEX does not spend the full approved amount during a swap.

If a DEX leaves a non-zero residual allowance (“dust allowance”), the post-swap check may fail or cause unexpected behavior. While the currently supported DEXs fully consume allowances and work as expected, this logic introduces fragility when integrating new DEXs.

Recommendation

Exercise caution when integrating new DEXs. Ensure they either fully consume the approved allowance or update the post-swap logic to safely handle residual allowances. Alternatively, consider explicitly resetting allowances after swaps instead of relying on full consumption.

Status

Acknowledged

Response: Leaving as is, but noted that DEXs must be vetted carefully.

Gas

Gas 01 Insufficient `msg.value` Will Lead To Wasted Gas Due To Missing Early Check

Location

BaseAggregator.sol

Description

The contract makes external calls to swap aggregators without first validating that the ETH value sent with the transaction (`msg.value`) is sufficient to cover protocol fees. In functions like `fillQuoteEthToToken`, `_fillQuoteTokenToEth`, and `_fillQuoteTokenToToken`, the code performs multiple operations before the external call:

1. Warrant signature verification
2. Initial balance snapshots
3. Token transfers and approvals
4. Other state-modifying operations

Only after all these operations does the code execute the external call to the aggregator. If `msg.value` is insufficient for the aggregator's protocol requirements, the call will revert. However, the contract will have already consumed significant gas executing all the preceding checks and state changes, resulting in wasted gas for a transaction that was doomed to fail from the start.

Recommendation

Add an early minimum `msg.value` validation check at the beginning of each function that accepts ETH. For example:

```
1 function fillQuoteEthToToken(  
2     address buyTokenAddress,  
3     address payable target,  
4     bytes calldata swapCallData,  
5     uint256 feeAmount,  
6     CanoeHelper.Warrant calldata warrant  
7 ) external payable whenNotPaused nonReentrant ... {  
8     require(msg.value ≥ MIN_ETH_VALUE, "INSUFFICIENT_ETH_VALUE");  
9  
10    // ... rest of function  
11 }
```

This validates the transaction's viability early, preventing gas waste on calls that will inevitably fail due to insufficient ETH.

Status

Fixed

Response: Added a check in `fillQuoteEthToToken` to ensure `msg.value > feeAmount`.

Gas 02 Redundant Recovered Signer Address Zero Check

Location

CanoeHelper.sol:78

Description

The `verifyWarrant` function contains a redundant check that validates `recoveredSigner ≠ address (0)`. This check is ineffective due to two complementary reasons:

1. `ECDSA.recover()` behavior: The OpenZeppelin `ECDSA.recover()` function does not return `address(0)` for invalid or malformed signatures. Instead, it reverts/throws an exception if the signature is invalid. Therefore, if the function successfully returns from `ECDSA.recover()` without throwing, the recovered address is guaranteed to be non-zero.
2. Early exit logic: At the very beginning of the `verifyWarrant` function, there is an early return statement:

```
1 if (warrant.verifyingSigner == address(0)) {  
2     return;  
3 }
```

This means if `verifyingSigner` is `address(0)`, execution never reaches the `ECDSA.recover()` call or the subsequent validation checks. Therefore, there is no logical code path where `recoveredSigner` could simultaneously be `address(0)` AND the function continue execution past the first check.

The combination of these two factors makes the `require(recoveredSigner != address(0), "CANOE: INVALID_SIG_RECOVERY");` statement unreachable under normal execution conditions, causing unnecessary gas consumption without providing any actual security benefit.

Recommendation

Remove the redundant check entirely:

```
1 // 4. Recover the signer  
2 address recoveredSigner = ECDSA.recover(  
3     digest,  
4     warrant.signature  
5 );  
6  
7 // 5. Validate the signer  
8 // Removed redundant check: require(recoveredSigner != address(0), "CANOE:  
9     INVALID_SIG_RECOVERY");  
9 require(  
10     recoveredSigner == warrant.verifyingSigner,  
11     "CANOE: INVALID_SIGNATURE"  
12 );
```

This removes the dead code path and saves gas on every warrant verification without sacrificing any security, since:

- Invalid signatures will cause `ECDSA.recover()` to revert before reaching the remaining checks
- The subsequent equality check against `warrant.verifyingSigner` provides the actual validation needed

Status

Fixed

Gas 03 Missing Permit Deadline Pre-Check

Location

PermitHelper.sol

Description

In the EIP-2612 permit flow, the contract directly calls `IERC2612(tokenAddress).permit( ... )` without first validating that the permit has not expired.

If `permitData.deadline` has already passed, the external call will revert, wasting gas on an avoidable failure.

Recommendation

Add an explicit deadline check before calling `permit`:

```
1 require(block.timestamp ≤ permitData.deadline, "PERMIT_EXPIRED");
```

This prevents unnecessary external calls when the permit is already invalid, saving gas and failing early with a clearer error condition.

Status

Acknowledged

Response: Leaving as is, in order for the extra check to make sense, a large number of input errors would need to occur, which indicates a front end error.